



REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: Ingeniería en informática
UNIDAD CURRICULAR: Ingeniería del software

Ensayo crítico: El riesgo de la inutilidad del software

NOMBRE DEL DOCENTE

Félix Márquez

NOMBRE DEL ESTUDIANTE

N.º C.I. 30.791.551

SECCIÓN: 2

CIUDAD GUAYANA, 02-09- 2026

Índice

Introducción	3
Desarrollo	4
Planteamiento	4
De la crisis técnica a la crisis de valor	4
Por qué importa la ingeniería de requisitos	5
Por qué importa identificar bien a los actores	5
Conclusión	6
Referencias Bibliográficas	7

Introducción

El presente informe desarrolla un ensayo crítico en torno al Ejercicio 4 propuesto en la Unidad 1 de la asignatura Ingeniería de Software II, cuya reflexión parte de la siguiente afirmación: “el mayor riesgo no es que el código no compile, sino que el software se construya perfectamente, pero nadie lo use”. A partir de esta premisa, se analiza cómo la ingeniería de requisitos, la identificación de actores (stakeholders) y la evolución histórica de la crisis del software explican por qué la inutilidad de un producto de software constituye, en la práctica moderna, un riesgo más grave que el propio fallo técnico.

Desarrollo

Planteamiento

Cuando pensamos en que un proyecto de software “falla”, normalmente imaginamos algo técnico: un error de compilación, un servidor caído, un plazo que no se cumplió. Pero la afirmación que estamos discutiendo apunta a otra cosa: un equipo puede hacer todo bien —código limpio, buena arquitectura, entregas puntuales— y aun así fracasar, simplemente porque nadie usa lo que construyeron.

Y ahí está el punto: eso no es un problema de programación. Es un problema de dirección del proyecto.

De la crisis técnica a la crisis de valor

La “Crisis del Software” se nombró así en una conferencia de la OTAN en 1968, para describir proyectos que se salían de presupuesto, se atrasaban y entregaban software de mala calidad. En ese momento el problema era básicamente de capacidad: la industria todavía no sabía construir software de forma confiable. Con el tiempo, pasamos de Cascada a Ágil y luego a DevOps, y hoy en día ya podemos desplegar código funcional en minutos, de forma bastante confiable. “El código no compila” ya casi no es lo que hunde un proyecto.

Pero la crisis no se resolvió, cambió de forma. Ahora ya no fallamos tanto en construir el software, fallamos en decidir qué construir. A esto se le llama a veces el “síndrome de la fábrica de funcionalidades”: equipos que entregan sprint tras sprint, muy productivos, pero que nunca se detienen a preguntar si lo que están haciendo realmente resuelve algo. Un proyecto puede cumplir perfecto con alcance, costo y tiempo, y aun así ser un fracaso total como producto, porque esas tres métricas miden que se ejecutó bien el plan, no que el plan sirviera para algo.

Por eso ayuda separar Gestión de Proyectos de Gestión de Productos. La primera se conforma con entregar lo que se prometió (el output). La segunda pregunta algo más incómodo: ¿esto generó algún impacto real, lo adoptó la gente, aportó valor? (el outcome). El riesgo del que habla el enunciado solo se nota cuando dejamos de preguntarnos “¿lo entregamos?” y empezamos a preguntarnos “¿alguien lo está usando?”.

Por qué importa la ingeniería de requisitos

La mayoría de las veces, este problema empieza mucho antes de escribir la primera línea de código: empieza en cómo se levantaron los requisitos.

Hay una diferencia entre la necesidad real de alguien y el requerimiento que terminamos escribiendo en el documento. Si esa traducción se hace mal —porque nadie preguntó bien, porque se asumió en vez de indagar, porque se anotó literalmente lo que el cliente dijo y no lo que en el fondo necesitaba— el equipo puede ejecutar con total precisión algo que apunta al lugar equivocado.

La Curva de Boehm dice que un error de requisitos detectado en mantenimiento cuesta hasta 100 veces más que si se detecta al principio. Pero en este caso el problema es todavía peor: si nadie usa el software, no hay mantenimiento que corrija nada, porque ya se perdió toda la inversión. Por eso estándares como el ISO/IEC/IEEE 29148 insisten en que un requerimiento no solo debe poder implementarse, también tiene que ser necesario y estar conectado a una necesidad real, no solo sonar técnicamente correcto.

Por qué importa identificar bien a los actores

El otro origen de este problema es no identificar bien a los actores del proyecto. Algo que suele olvidarse es que los stakeholders no son solo los que quieren que el proyecto salga adelante; también hay quienes prefieren que no se haga, o a quienes simplemente les da igual. Si el equipo solo le hace caso al que paga o al gerente que aprueba el presupuesto, y nunca llega al usuario que en verdad va a usar el sistema todos los días, es muy probable que termine construyendo lo que el patrocinador cree que hace falta, no lo que el usuario realmente necesita.

Esto es justamente uno de los errores clásicos que menciona el CHAOS Report: no involucrar al usuario, construir lo que el cliente cree que quiere en lugar de lo que necesita. Y da igual qué tan ágil sea el equipo, si entrega cada dos semanas o tiene 90% de cobertura de pruebas —nada de eso arregla el problema si desde el principio no se identificó bien para quién se estaba construyendo.

Conclusión

Un error de compilación se nota al instante, es molesto, pero hoy en día es barato de arreglar. El software que funciona perfecto pero que nadie usa es distinto: no lanza ninguna alerta, no rompe ningún build, no aparece en ningún dashboard. Se nota meses después, cuando ya se gastó el presupuesto y el esfuerzo de todo el equipo. Por eso, en la práctica, es el riesgo más grande: no es que el sistema falle, es que el sistema nunca tuvo sentido de existir.

La solución no es programar mejor, es gestionar mejor todo lo que pasa antes de programar: levantar requisitos que reflejen necesidades reales, y no solo lo que alguien pidió de pasada, y mapear con cuidado a todos los actores involucrados, no solo a quien firma el contrato. Al final, la ingeniería de software moderna no se trata de escribir el código más elegante posible, se trata de asegurarse de que, a alguien, de verdad, le importe.

